

APR Design Flow 重點整理

Design Import → Sign-off

各階段要做什麼、要注意什麼

Agenda

1. 九大步驟總覽
2. Step 1–3 : Design Import → Floorplan → Power Planning
3. Step 4–6 : Placement → CTS → Routing
4. Step 7–9 : DFM → Verification / Sign-off → Data Export
5. 總結：檢查清單 + 貫穿全流程的疊代收斂循環

九大步驟總覽

#	階段	一句話重點
1	Design Import	匯入網表 / 約束 / 製程檔，建 MMMC
2	Floorplan	定晶片尺寸、cell utilization
3	Power Planning	建電源 ring / stripe / rail
4	Placement	擺巨集與標準單元、時序收斂
5	CTS	合成時脈樹、平衡 skew
6	Routing	訊號線繞線、修 DRC / antenna
7	DFM	Filler、metal fill 收尾
8	Verification / Sign-off	DRC / LVS / timing 全部過關
9	Data Export	輸出 GDS / SDF / 網表交付

資料來源

- gcd：90nm 教學範例，九步驟各跑一次，求簡化
- DTMF_CHIP：180nm 真實課程作業，每個階段都反覆疊代很多輪——本簡報的「真實案例教訓」都來自這裡

後面每個 Step 的指令與數字，都是從這兩個實際案例的 checkpoint / 指令歷史直接還原，不是憑空舉例。

Step 1–3：從匯入到電源網路

Step 1 : Design Import — 要做什麼

匯入合成後網表、SDC 時序約束、LEF/Lib 製程檔，建立 MMMC（多角多模）分析設定。

```
loadConfig ${TOP_DESIGN}.conf 1
```

一份 `.conf` 通常集中定義：netlist、SDC、LEF、IO 檔、MMMC view 檔。

Step 1 : Design Import — 要注意什麼

- 匯入後不能有 **load error**——網表 / 約束 / 製程檔版本要對得上
- MMMC 至少要涵蓋 **setup**（慢角）與 **hold**（快角）兩個 view，不能只用一組 corner
- MMMC 設定一旦建立，**全程沿用到 Route 結束**，不會每階段重建

```
create_delay_corner -name corner_max -library_set libs_max -rc_corner rc_corner    ;# 慢角=setup  
create_delay_corner -name corner_min -library_set libs_min -rc_corner rc_corner    ;# 快角=hold  
set_analysis_view -setup {view_setup} -hold {view_hold}
```

setup 用慢角（延遲最長）、hold 用快角（延遲最短），符合「setup 抓最壞情況、hold 抓最壞情況」的物理直覺——這組設定要先建對，後面所有時序分析都靠它。

Step 2 : Floorplan — 要做什麼

決定 die / core 尺寸、長寬比、cell utilization，評估巨集擺放位置。

```
floorPlan -r 0.75 0.7 100 100 100 100
```

-r 格式：aspect ratio、cell utilization、core 到 IO 四邊間距。

Step 2 : Floorplan — 要注意什麼

- Cell utilization 通常抓 **60–80%**——太高繞不進去，太低浪費面積
- 巨集盡量靠 die 邊緣、對齊擺放，中間留給標準單元與繞線通道
- 尺寸不是一次到位，通常先抓大概數字，看 placement / routing 結果再調整

```
floorPlan -r 0.73 0.7 100 100 100 100      ;# 先試算  
floorPlan -r 0.75 0.702385 100.94 100.44 100.32 100.24 ;# 看過壅塞/時序後再定案
```

真實案例：DTMF_CHIP 就是先用一組粗略數字試算，確認可行後才微調到最終版——floorplan 尺寸幾乎沒有人第一次就抓準。

Step 3 : Power Planning — 要做什麼

建立電源 ring（環）→ stripe（網格）→ rail（標準單元電源軌），三層電源網路。

```
addRing -nets {VDD VSS} -type core_rings -layer {top M5 bottom M5 left M6 right M6} -width 7
```

Step 3 : Power Planning — 要注意什麼

- `verifyConnectivity` 必須 0 error 才能進入下一階段（浮接電源會讓後面所有時序分析失真）
- 巨集（尤其 RAM / ROM / PLL 這類硬巨集）通常要額外加一圈 **block ring**，不能只靠 core ring
- Stripe 密度要夠——依 IR drop / EM 需求決定數量與寬度

```
addRing -nets {VDD VSS} -type block_rings -around selected    ;# 巨集額外包一圈  
verifyConnectivity -type all -error 1000 -warning 50           ;# 收尾前一定要跑
```

Step 3：解決手法 — IR Drop 太大怎麼辦

電源網路上某處電壓降（IR drop）太大，會拖慢附近邏輯時序，甚至造成誤動作。

- 加密 **stripe**：密度不夠是最常見原因，補幾條上去
- 加寬 **stripe / rail** 線寬：線越粗電阻越小，電壓降自然變小
- 針對熱點區域優先補強（例如巨集附近耗電特別集中的地方）

```
addStripe -nets {VDD VSS} -layer M5 -width 10 -set_to_set_distance 200 ;# 加密/加寬
```

Step 4–6：擺放、時脈、繞線

Step 4 : Placement — 要做什麼

擺放標準單元與（未固定的）巨集，時序驅動＋壅塞驅動雙重最佳化，跑 pre-CTS 時序最佳化。

```
place_opt_design
```

Step 4 : Placement — 要注意什麼

- Congestion 熱圖有沒有大面積熱點——壅塞會拖累後面 routing
- Pre-CTS setup WNS 要轉正，DRC 要乾淨，才能進 CTS
- 有 scan chain 的設計記得 `reorderScan`，避免掃描鏈繞線暴長

```
specifyScanChain scan1 -start IOPADS_INST/scanin/C -stop IOPADS_INST/scanout/I  
setPlaceMode -congEffort high -timingDriven 1 -reorderScan 1
```

真實案例教訓：DTMF_CHIP 這輪 placement 反覆跑了 5 次 `place_opt_design` 才收斂——placement 幾乎不會一次到位。

Step 4：解決手法 — Congestion 太高怎麼辦

Congestion 熱圖出現大面積紅色熱點，代表這區要繞的線比可用資源多，繞不進去。

- 提高 congestion effort，重跑 placement，讓工具更積極分散 cell
- 調整巨集擺放位置 / 方向，留出繞線通道（避免巨集把走道全部擋死）
- 壅塞太嚴重時要回頭調整 **floorplan**（加大 core、降低 utilization），不要硬凹 placement

```
setPlaceMode -congEffort high  
place_opt_design -incremental
```

Step 5 : CTS — 要做什麼

指定 clock buffer / inverter cell → 建 clock tree spec → 實際蓋出時脈樹、平衡 skew。

```
cconst_design
```

Step 5 : CTS — 要注意什麼

- **Hold time** 要等 CTS 做完才能精確算——CTS 前的 hold 檢查都不準
- 除了 skew，也要看 **DRV** (max transition / max capacitance / max fanout) 有沒有超標
- 特定 clock domain 有問題時，可以只針對它做「by-item」局部重跑，不用整個 CTS 重來

```
timeDesign -postCTS -drvReports -slackReports -outDir timingReports  
timeDesign -postCTS -hold -slackReports -outDir timingReports
```

-drvReports 把 DRV 違規 (max cap / tran / fanout) 另外拉一份報告，跟時序分開看。

Step 5：解決手法 — DRV 怎麼修

- `cchopt_design` 本身就會自動調整 buffer 尺寸、插入額外緩衝級來修 DRV，不用手動介入
- 時脈這種高扇出訊號，還可以套用 **NDR（非預設走線規則）**：加寬線寬降低電阻、加大線距減少串擾
- 修完記得重新跑一次 `timeDesign -drvReports` 確認真的清零

```
set_clock_tree_options -routing_rule my_ndr ;# 時脈套用加寬線寬/加大線距的規則  
cchopt_design -cts ;# 重跑一次，順便修 DRV
```

Step 6 : Routing — 要做什麼

Global routing（分格子、抓大致路徑）→ Detail routing（畫出精確金屬線與 via）。

```
routeDesign -globalDetail -via0pt -wire0pt
```

Step 6 : Routing — 要注意什麼

- Routing 是「繞線 → 檢查 DRC / antenna → 再繞線」不斷疊代，不是跑一次就結束
- **Antenna 違規**（金屬走線過長累積電荷）要收斂到 0，通常靠跳層修復
- 大量疊代後若時序嚴重劣化，代表 placement / CTS 需要回頭調整，不要死磕 routing
- 少數違規路徑可以用**局部 ECO** 修正，不用整個階段重跑

```
verifyProcessAntenna -report DESIGN.antenna.rpt -error 1000  
ecoChangeCell -inst <hold_violating_reg> -downsize ;# 局部修 hold，不動其他已收斂部分
```

真實案例教訓：DTMF_CHIP 全流程 routeDesign 系列指令共呼叫 **103 次**、antenna 檢查呼叫 **27 次**——這是常態，不是設計出了問題。

Step 6：解決手法 — Antenna 違規怎麼修

金屬走線過長，蝕刻過程中會像天線一樣累積電荷，打穿閘極氧化層。兩種常見修法：

- **跳層 (layer jumping)**：讓連到閘極的線提早跳到上層金屬，蝕刻低層時暴露面積變小——對既有繞線改動最小，router 預設就是用這招自動修
- **插二極體 (diode)**：在受影響的網路上接一顆反向二極體到 GND，把多餘電荷導走，但需要額外空間放二極體 cell

```
routeDesign -globalDetail -via0pt ;# 重跑一次，router 會自動嘗試跳層修 antenna
```

Step 7–9：收尾、驗收、交付

Step 7 : DFM — 要做什麼

插入 filler cell 填滿 cell row 空隙，插 dummy metal fill 滿足金屬密度規則。

```
addFiller -doDRC  
addMetalFill
```

Step 7 : DFM — 要注意什麼

- Filler 要維持 N/P well 與電源軌連續，不能留空隙
- 量產設計還會插 **well tap**（防 latch-up）、**end cap**（保護 row 邊界）、**decap**（穩壓）
- 這些不是每個練習案例都會做，但正式產品線通常缺一不可

```
setEndCapMode -boundary_tap false    ;# 例：DTMF_CHIP 關掉了 endcap 順便當 well tap 用的功能
```

這行是真實案例裡找到的設定——代表這個練習沒有做 well tap / end cap 收尾，量產設計要記得補上。

Step 8 : Verification / Sign-off — 要做什麼

三項驗證缺一不可：**DRC**（幾何規則）、**LVS 概念**（連接性）、**Antenna**（天線效應），加上 timing sign-off。

```
verifyGeometry  
verifyConnectivity -type all
```

Step 8 : Verification / Sign-off — 要注意什麼

- Sign-off 用的分析設定比平時疊代**更嚴謹**——要開 **OCV derate**、用真實寄生參數，不能只用平時疊代的寬鬆設定
- 嚴謹的產線流程還會**換一套獨立工具**重新檢查一次（DRC 用 Hercules / Calibre、timing 用 PrimeTime），不能只信任 P&R 工具自己的估算
- Hold 幾乎壓線時，下一步通常是針對那條路徑做局部 `optDesign -postRoute -hold`，不用整個階段重跑

```
setAnalysisMode -analysisType onChipVariation    ;# 真實案例只在 Route 階段才第一次開啟  
optDesign -postRoute -hold                      ;# 精修壓線的 hold 路徑
```

Step 8：解決手法 — Setup / Hold 怎麼修

- **Setup 違規**（訊號到得太慢）：`-upsized` 換驅動力更強的同功能 cell，或插 buffer 減少長線負載延遲
- **Hold 違規**（訊號到得太快）：`-downsize` 換驅動力較弱、延遲較大的 cell，或插 buffer 墊高延遲
- 兩者都是**局部 ECO**：只動違規那幾顆 cell，其餘已收斂的部分完全不受影響，不用整個階段重跑

```
ecoChangeCell -inst <late_reg> -upsized      ;# 修 setup  
ecoChangeCell -inst <early_reg> -downsize    ;# 修 hold
```

Step 8 : DTMF_CHIP 最終 Sign-off 結果

檢查項	結果
DRC	0
verifyConnectivity	0 errors
verifyProcessAntenna	0 violations
Setup WNS	0.022 ns (0 違規)
Hold WNS	-0.000 ns (1 條微幅違規)

三項驗證 + timing sign-off **全部通過才算完成**；hold 幾乎壓線，代表這種結果背後往往需要一輪局部 ECO (downsize / upsize cell) 才能收斂。

Step 9 : Data Export — 要做什麼

設定分析模式（如 `bcwc`），輸出 SPEF（真實寄生參數）、SDF（延遲）、post-APR 網表、GDSII 版圖、LEF abstract。

```
streamOut ${TOP_DESIGN}.gds -mapFile ... -mode ALL
```

Step 9 : Data Export — 要注意什麼

- SPEF 是給獨立 sign-off STA 工具用的**真實**寄生資料，不是估算值
- GDSII 是實際送晶圓廠的檔案，輸出前務必確認前面所有 sign-off 都已通過
- 交付清單：GDS / SDF / SPEF / post-APR 網表 / LEF abstract 缺一不可

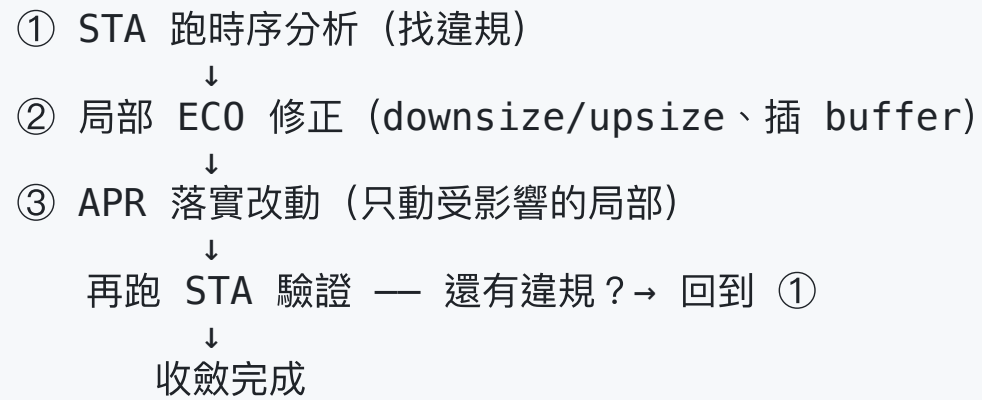
```
rcOut -spef_file ${TOP_DESIGN}.spef  
write_sdf ${TOP_DESIGN}.sdf
```

總結

各階段檢查清單總表

#	階段	完成判斷
1	Design Import	無 load error，MMMC 涵蓋 setup/hold
2	Floorplan	面積、utilization、aspect ratio 合理
3	Power Planning	verifyConnectivity 0 error
4	Placement	Pre-CTS WNS 轉正、DRC 乾淨
5	CTS	Post-CTS setup/hold 收斂、DRV 無超標
6	Routing	DRC = 0、antenna = 0
7	DFM	密度規則通過
8	Verification	DRC / LVS / Antenna / Timing 全過
9	Data Export	交付檔案齊全（GDS/SDF/SPEF/網表）

貫穿全流程的一件事：疊代收斂循環



為什麼這個循環重要

九個步驟裡每一次「反覆疊代」——Placement 5 輪、Routing 103 次呼叫、CTS by-item 微調——本質上都是同一個循環：

跑分析 → 找問題 → 局部修正 → 再驗證，不斷重複直到全部違規清零。

真實 APR 流程不是九步驟走一次就結束，而是「做完 → 檢查 → 不合格就疊代重做」的持續循環。

Thank You

Q&A